



Figure 10: Output of the list using JSX and React



Figure 11: *AddItem* component

The basic example is very simple, but when updating many elements on pages or adding elements, the created elements will be difficult to maintain. React allows you to create reusable components using JSX.

JSX stands for JavaScript XML. It allows users to create JavaScript objects using the HTML syntax. The JSX approach is that the markup and code that generates it are tied together. Using a templating system will make the display logic more complex and cumbersome.

JSX syntax needs to be converted into JavaScript before it runs in the browser. Babel is a tool that will compile JSX into JS. We need to add a link for Babel in the script. We also need to set the type as *text/babel*.

Let us modify the above example to make it a component and use JSX instead of *createElement*. We will create a class which will have the render function. We will store it in a variable *HelloWorld*, making it a component that can be used later in *React.render*. Figure 6 shows the new updated code with JSX.

The output is shown in Figure 7.

Let's now add the properties to our components.

Properties are nothing but attributes to an HTML. In the React class, these can be accessed using *this.props*, and a change in properties will render the component again. Let us add the property *uname* to display *Hello* followed by *uname*. Figure 8 shows the modified code.

Refresh the browser, or open the modified file in a browser. This will display *Hello* followed by a name as shown in Figure 9.

We can separate the JSX code in a different JavaScript file. We will try to build a simple list that will display the data that has been entered in a text box, in a bulleted list.

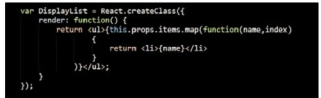


Figure 12: *DisplayList* component

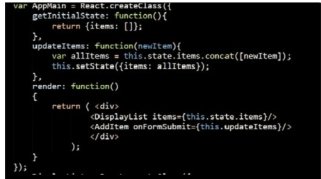


Figure 13: *AppMain* component

The React approach is to view everything as components, breaking them down in the form of a UI hierarchy. Apart from properties, components can handle events and save states too, which we will look at in the next example.

For this application, we need the following three components:

- Form for input: *AddItem* component
- List to display data: *DisplayList* component
- Main component to hold the above two: *AppMain* component

The output of the application is shown in Figure 10.

The code for *AddItem* is shown in Figure 11,

DisplayList is shown in Figure 12 and *AppMain* is shown in Figure 13.

Changes in properties and the state will trigger updates and rendering, which in turn will update the component state. Properties and states are both used to hold some information in the form of JavaScript objects, which are used in the display. The major difference is that the state starts with a default value and changes value in due course. To set the state, first the *getInitialState* method is used.

This is a simple demo of how to use React JS. For more details, refer to the online documentation at <https://facebook.github.io/react/docs/getting-started.html>. More examples can also be found if you download the starter kit from the download page. 

By: Ashish Singh Bhatia

The author is a technology enthusiast and a FOSS fan. He loves to explore new technology and work on Python and Java languages. He can be reached at ast.bhatia@gmail.com and he blogs at <https://openfreedea.wordpress.com/>.